

CRUNCH FORCE 04 · PADS Datathon — Sprint 1

Technical Challenges & Solutions

TOPIC	TECHNICAL CHALLENGE	SOLUTION
Multi-Format JSON Ingestion	The dataset arrived in four completely different formats with no unified schema or relationship documentation. Patient, Questionnaire, and Observation data were spread across multiple JSON files in separate folders. Time-Series data existed as thousands of raw .txt sensor files with no direct Power BI ingestion path. There were no pre-defined keys, no shared column naming conventions, and no inter-table relationship map.	Performed systematic dataset profiling before any transformation to understand structure, volume, and relationships. • Patient → primary dimension (anchor of the model) • Questionnaire → indirect dimension via Patient ID • Observation → session-level dimension • Time-Series → high-volume fact source needing pre-processing Used Power BI folder ingestion: Home → Get Data → Folder → Combine & Load for all JSON tables — auto-detected and merged all files.
High-Freq Sensor ETL Pipeline	Each patient session produced a separate .txt file with high-frequency accelerometer and gyroscope readings (X, Y, Z axes). Across all participants and tasks, this resulted in millions of sensor rows — direct Power BI ingestion would have caused severe performance degradation and made DAX calculations practically unusable. Additionally, activity type and device location were not structured columns — they were embedded inside the source filenames.	Engineered a 3-stage Python pre-processing pipeline: Stage 1 — Text to Parquet: All .txt files read via pandas, combined with concat(), serialised to Parquet for fast Power BI ingestion. Stage 2 — Aggregation & Feature Engineering: Activity (CrossArms, Walking, FingerTapping) and device location (LeftWrist, RightWrist) extracted from filenames. Grouped by subject_id + activity + device_location. Computed mean Accel_X/Y/Z and Gyro_X/Y/Z. Result: Millions of rows → 10,318 clean aggregated records. Stage 3 — Power BI: PatientID cast to Whole Number; sensor values rounded to 6 decimal places.

TOPIC	TECHNICAL CHALLENGE	SOLUTION
Power Query ETL & Data Quality	The Patient table had multiple data quality issues: • Height outlier: 55 cm (should be 155 cm) • Missing disease comments across all 3 condition groups • Clinical abbreviations in free-text fields (IPS, THS, MSA, PSP, DD, SWEDD, SPMS, ET, PSP-RS) — unexpanded • Inconsistent casing & whitespace in Gender, Handedness, Age, Height, Weight • Duplicate Patient IDs • Constant system columns inflating model size (resource_type, study_id, Source.Name)	Implemented an 8-step Power Query ETL pipeline: • Ingestion: JSON folder load — auto-combined all Patient JSON files • Noise Removal: Dropped resource_type, study_id, Source.Name; removed duplicate Patient IDs • Field Renaming: ID → Patient ID; standardised all column labels • Value Standardisation: Title Case on all fields; Gender → Male/Female; Handedness → Right/Left • Comment Cleaning: Trim + Clean + Title Case; expanded all abbreviations • Transformation: Condition grouped → PD DD Healthy • Imputation: Missing comments filled with 'No Comments' • Validation: Height 55 → 155 cm; all data types assigned correctly
Noise Reduction & Type Casting	The Questionnaire table (14,070 rows) contained 5 constant, redundant columns that added zero analytical value but inflated model size. The Observation table contained 7 redundant/constant system fields including hardware specifications (bits, sampling_rate) and file metadata. Both tables had inconsistent column naming conventions that needed alignment with the Patient table for correct joins.	Questionnaire: • Dropped: resource_type, study_id, id, questionnaire_name, Source.Name • Renamed: Text → Question; subject_id → Patient_id • Types: Patient_id → Whole Number; Question, Answer → Text; Timestamps → Date/Time Observation: • Dropped 7 fields: Name, resource_type, study_id, data_type, bits, sampling_rate, file_name • Consistent casing, trimming, and cleaning on all text fields • Types: Session IDs → Whole Number; Measurements → Decimal; Timestamps → Date/Time

TOPIC	TECHNICAL CHALLENGE	SOLUTION
Snowflake Schema & Dimensional Modelling	A naïve star schema would have incorrectly linked Dim_Questionnaire directly to Fact_Timeseries, producing semantic inaccuracies — questionnaire responses belong to patients, not to individual sensor timestamps. Designing correct cardinalities and surrogate keys for the Observation table (same patient, multiple sessions) required deliberate key engineering. The wrong schema choice would have caused incorrect DAX aggregations and inflated fact table cardinality.	Implemented a Snowflake Schema with 4 tables: • Fact_Timeseries — Central hub: aggregated sensor metrics (10,318 rows) • Dim_Patient — Primary dimension: demographics, diagnosis, clinical metadata • Dim_Observation — Session dimension: device location, activity, Observation_Key • Dim_Questionnaire — Indirect dimension: linked via Dim_Patient only (NOT the fact table) Relationships: • Dim_Patient 1:Many → Fact_Timeseries (Patient ID) • Dim_Observation 1:Many → Fact_Timeseries (Observation_Key) • Dim_Patient 1:Many → Dim_Questionnaire (Patient ID) — Snowflake link
DAX Measures & Context Transition	Raw aggregated data alone was insufficient for clinical insight. Derived metrics for condition distributions, motor task comparisons, demographic analytics, and questionnaire scoring were all needed. DAX measures had to correctly traverse the indirect Snowflake path (Fact → Dim_Patient → Dim_Questionnaire), which is more complex than standard star schema DAX. 30+ correct, performant DAX queries were required to cover the full analytical scope.	Authored 30+ DAX measures and calculated columns covering: • Condition Distribution: Patient counts & % per group (PD / DD / Healthy) • Sensor Metrics: Mean Accel & Gyro X/Y/Z by condition, activity, device • Motor Task Comparisons: CrossArms, Walking, FingerTapping, HandsAtRest • Demographics: Age distribution, gender breakdowns, handedness analysis • Questionnaire Scoring: DAX traversing Fact → Dim_Patient → Dim_Questionnaire • Condition Filters: Atypical Parkinsonism, Essential Tremor, MS subtypes • Device Comparisons: LeftWrist vs. RightWrist signal differentials All measures validated for correct context transition across Snowflake relationships.

TOPIC	TECHNICAL CHALLENGE	SOLUTION
Python Visual Integration in Power BI	Standard native Power BI visuals were insufficient for clinical biomarker analysis. Multi-dimensional motor comparisons, patient flow mapping, inter-axis correlation matrices, and condition-level profiling all required chart types not available in the native Power BI visual library. Each visualisation demanded custom Python scripting embedded inside Power BI via the Python Visual canvas element, with data passed directly from the Power BI data model into the script.	Derived 4 advanced Python visualisations using matplotlib, numpy and pandas: • Radar Chart: Multi-axis overlay of mean Accel & Gyro metrics across PD / DD / Healthy groups — simultaneous comparison of all 6 sensor dimensions per condition group • Heatmap: Correlation matrix of accelerometer and gyroscope axes (X/Y/Z) per activity and device location — built with matplotlib and numpy to reveal inter-axis signal patterns • Sankey Diagram: Patient flow mapping from condition groups to disease subtypes to device locations — visualising data distribution across Snowflake model layers • Scatter Plot: Accel vs. Gyro mean values per patient, colour-coded by condition group — identifies motor signal clusters and outliers across PD, DD, and Healthy cohorts All scripts used pandas for in-context data reshaping before rendering.

Power BI (Power Query + DAX) | Python (pandas, matplotlib, seaborn, scipy) | Snowflake Schema | 10,318 aggregated rows | 30+ DAX measures